Speculative Decoding and Inference Acceleration

pig7selene

September 5, 2026

Abstract

Autoregressive Decode exposes only one new target-model token at a time, even when the target model can score a short known continuation in parallel. Speculative Decoding uses a cheaper Draft Model to propose such a continuation, asks the Target Model to verify it in one pass, and commits only the target-authorized prefix. This chapter derives the acceptance correction that makes stochastic speculative sampling distribution preserving, develops a simple expected-speed model, and examines self-speculation, multi-token prediction, cache state, and serving consequences. The aim is to distinguish an exact acceleration technique from an uncontrolled approximation.

1 Introduction

Chapter 23 describes ordinary Decode as a feedback loop: the Target Model produces next-token logits, a decoding rule selects one token, and that token becomes the input to the next Target Model step. Even with a KV Cache, a continuation of N tokens normally requires N serial Target Model invocations. Chapter 24 reduces the cache memory that this loop consumes, Chapter 25 reduces the weight footprint, and Chapter 26 decides which requests may take a Decode step. None of those changes removes the token-to-token dependency itself.

Speculative Decoding changes the unit of useful Target Model work. It first lets a cheaper Draft Model propose a short continuation, then evaluates the whole proposed continuation with the Target Model under causal attention. If several proposed tokens agree sufficiently with the Target Model, one Target Model verification pass commits several output tokens. The Target Model remains authoritative: a draft token is never final merely because the Draft Model produced it.

This chapter focuses on draft-and-verify generation. It does not treat general model compression, distributed inference, or every method that predicts multiple candidate tokens. The central question is narrower: how can a system reduce the number of **serial** Target Model Decode iterations without changing the specified Target Model distribution?

2 The Sequential Decode Bottleneck

Let $c_t = \left(x, y_{\{<t\}}\right)$ be the prompt x together with the generated prefix before position t. Ordinary stochastic decoding samples

$$y_t \sim p_{\theta(\cdot \mid c_t)}, \tag{1}$$

where p_θ is the Target Model distribution after all declared temperature, truncation, and logit-transform rules have been applied. The selected y_t is required before c_{t+1} is known. Thus, a single request has little token-position parallelism during Decode, even though the Transformer can process the known prompt in parallel during Prefill.

This serial dependency is particularly costly when a Target Model's Decode step is dominated by reading its weights and the current KV Cache rather than by peak arithmetic throughput. A short **known** continuation can often be scored by one causal Target Model pass with less wall-clock cost than performing the same number of independent, sequential Decode iterations. Speculative Decoding exploits that distinction; it does not claim that causality has disappeared [1].

For the remainder of the chapter, let the Draft Model distribution be $q_{\varphi(\cdot \mid c_t)}$, where φ denotes its parameters. The Draft Model may be a separate small model, a reduced execution path through the Target Model, or an auxiliary prediction module. Its implementation varies, but its role is fixed: it proposes likely future tokens cheaply. The Target Model p_θ defines what may be committed.

3 Drafting and Parallel Verification

At one speculative iteration, the Draft Model samples or chooses up to γ proposed tokens $\tilde{y}_1, \ldots, \tilde{y}_\gamma$ autoregressively. It produces distributions

$$q_{i(v)} = q_{\varphi\left(v \mid c_t, \tilde{y}_{\{<i\}}\right)} \tag{2}$$

and draws $\tilde{y}_i$ from q_i for $i = 1, \ldots, \gamma$. The notation distinguishes a **proposed token** from a **verified** token: the former is an inexpensive prediction; the latter has passed the target-side acceptance rule.

The Target Model then consumes the known prefix together with the proposed block under its ordinary causal mask. In one verification pass, it computes

$$p_{i(v)} = p_{\theta\left(v \mid c_t, \tilde{y}_{\{<i\}}\right)}, \quad i = 1, \ldots, \gamma + 1. \tag{3}$$

The first γ distributions score the proposed positions; the final distribution supplies an additional next-token distribution if every proposal is accepted. The Target Model does not treat the proposals as facts. It evaluates their conditional probabilities exactly as it would evaluate an observed continuation, while causal masking prevents position i from reading later proposals.

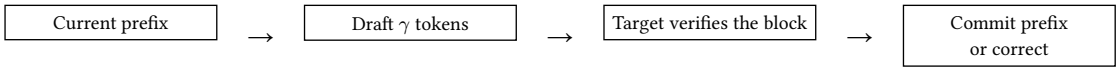

Figure 1: A speculative iteration converts a short draft continuation into Target Model work that can authorize multiple output tokens. The first rejected proposal terminates the candidate prefix.

The phrase **parallel verification** has a precise but limited meaning. The Target Model still respects causal dependencies within the candidate block. It can nevertheless evaluate the logits for all candidate positions in a single block-shaped forward pass, rather than waiting for an external sampling decision between each position. This is the execution opportunity shown in Figure 1.

4 Acceptance, Rejection, and Exact Sampling

For stochastic decoding, agreement cannot mean only that the Draft Model's most likely token matches the Target Model's most likely token. The two models generally define different categorical distributions. A naïve rule that commits every draft sample would sample from q_φ, not from p_θ, and would alter the Target Model's output distribution.

For proposed token $\tilde{y}_i$, exact speculative sampling accepts with probability

$$a_i = \min\left(1, \frac{p_{i(\tilde{y}_i)}}{q_{i(\tilde{y}_i)}}\right). \tag{4}$$

Equivalently, sample $u_i \sim \mathrm{Uniform}(0, 1)$ and accept when $u_i \leq a_i$. If $q_{i(\tilde{y}_i)} \leq p_{i(\tilde{y}_i)}$, the proposal is always accepted; where the Draft Model places more probability mass on a token than the Target Model does, acceptance corrects the excess. The procedure considers proposals in order and stops at the first rejection.

At a rejection position j, the system samples a replacement from the positive residual distribution

$$r_{j(v)} = \frac{\max\left(0, p_{j(v)} - q_{j(v)}\right)}{\sum_{w \in \mathcal{V}} \max\left(0, p_{j(w)} - q_{j(w)}\right)}. \tag{5}$$

It commits the previously accepted proposals followed by this replacement and discards every later proposal. If all γ proposals are accepted, it instead draws one additional token from $p_{\gamma+1}$. The resulting iteration always commits at least one token and at most $\gamma + 1$ tokens.

The correction in Equation 5 is what makes the method exact with respect to the declared Target Model sampling distribution. Intuitively, acceptance preserves the probability mass where the draft proposal is compatible with the target; residual sampling supplies precisely the target mass that the draft did not supply. This construction, introduced for Transformer generation by Leviathan, Kalman, and Matias and independently developed as speculative sampling for large language models, is distribution preserving up to the numerical behavior of the implementation [1], [2].

4.1 Greedy Decoding and Stochastic Sampling

For **greedy** decoding, a simpler deterministic rule is sufficient. The Draft Model proposes its argmax tokens, and the Target Model commits the longest prefix for which each proposal equals the Target Model argmax at that position. At the first mismatch, the Target Model argmax replaces the proposal. The output therefore matches ordinary greedy Target Model decoding, subject to the same deterministic numerical execution.

For **stochastic** sampling, exactness depends on the acceptance probabilities and residual correction above. In particular, p_i and q_i must use compatible vocabulary identities and the intended sampling transformation. If the service applies temperature, Top-k, Top-p, repetition penalties, or a token mask, it must define whether those operations produce the p_i and q_i used by the acceptance rule. Applying an incompatible transformation only to one side breaks the claimed distributional guarantee.

5 Acceptance Rate and Expected Speed

Let A be the number of accepted draft tokens in one speculative iteration, and let $N = A + 1$ be the number of committed tokens, including either a target-side correction or an additional target sample. If each proposal has an approximately independent mean acceptance probability α, then the expected number of committed tokens is

$$\mathrm{E}[N] \approx \frac{1 - \alpha^{\gamma+1}}{1 - \alpha}, \tag{6}$$

with the limiting value $\gamma + 1$ when $\alpha = 1$. This is an explanatory model, not a service-level guarantee. Acceptance probabilities vary by prompt, token position, sampling rule, and request; successive proposals are also correlated. Its value is that it makes the main trade-off visible: a high-quality Draft Model can convert one Target Model verification into several committed tokens, whereas a poor Draft Model commonly produces only the corrective target token.

Let C_{target} be the cost of one ordinary Target Model Decode iteration and let one Draft Model step cost cC_{target}. Under the further idealization that verifying γ positions costs approximately C_{target}, a rough speedup model is

$$S_{\mathrm{ideal}} \approx \frac{1 - \alpha^{\gamma+1}}{(1 - \alpha)(1 + \gamma c)}. \tag{7}$$

Equation 7 exposes why Draft Model quality alone is insufficient. A larger Draft Model may raise α, but it also raises c. Increasing γ increases the maximum number of tokens a Target Model pass can commit, but also creates more draft work and more candidate suffix that will be discarded after an early rejection. Verification cost itself can grow with candidate length, context length, batching, kernel choice, and cache layout. Real speedup must be measured end to end on the intended workload, not inferred only from acceptance rate.

Quantity	Favorable condition	Why it matters
Acceptance rate α	Draft and Target distributions are close on the workload	More proposed tokens are committed per verification
Draft cost c	Drafting is substantially cheaper than target Decode	Proposal work does not consume the saved latency
Draft length γ	Long enough to exploit accepted prefixes, short enough to avoid waste	Sets both the maximum gain and the rejected-suffix cost
Verification execution	The target can score a short block efficiently	Parallel scoring must be cheaper than equivalent serial Decode steps

Table 1: Speculation pays only when accepted target-authorized work outweighs drafting and verification overhead. The variables interact; optimizing one in isolation can reduce end-to-end speed.

The practical choice of γ is therefore workload dependent. Simple, predictable continuations may support longer drafts; difficult or high-entropy continuations may reject early. Adaptive draft lengths can improve utilization, but they add policy state and need validation under the same latency objectives as Chapter 26. As Table 1 shows, a method with a high mean α can still disappoint if its Draft Model, verification kernel, or batching behavior is expensive.

6 Related Drafting Designs

The classical design uses a distinct Draft Model, often smaller than the Target Model but trained with compatible tokenization and formatting. This separation can give a very low c, but it adds model weights, cache state, deployment versioning, and an alignment problem: the Draft Model must remain useful on the Target Model's workload.

Self-Speculative Decoding instead derives a cheap draft path from the Target Model itself. A system may skip selected intermediate layers, exit early, or use a smaller internal predictor while retaining the full path for verification. Draft & Verify is one example: it selectively skips layers to produce draft tokens, then uses the unmodified model to validate the block [3]. Self-speculation can avoid a separately resident Draft Model, but its reduced path must still be cheap enough and accurate enough to justify the additional control flow. It is not exact merely because it shares parameters; exactness comes from target-side verification and correction.

Multi-Token Prediction (MTP) trains or attaches multiple prediction heads so that a shared model representation can propose several future positions. In one formulation, heads predict several offsets rather than only the immediate next token; the candidates may then be verified by a causal Target Model path. MTP can support decoding acceleration, but it is not identical to a small-model speculative scheme: its proposal mechanism is internal and its training objective or heads may be specialized for lookahead. Gloeckle et al. study multi-token prediction as an auxiliary training task, while Medusa uses additional decoding heads and tree-structured verification to create candidate continuations [4], [5]. Any quality-preservation claim still depends on the particular verification rule.

7 KV Cache, Batching, and Serving

Speculation introduces provisional state. The Target Model must evaluate the candidate block with the same positional convention and attention semantics as ordinary Decode, which usually creates tentative target-side KV entries for the proposed positions. After verification, the implementation commits only the accepted prefix and the target correction, then discards or rolls back the rejected suffix. A Draft Model has its own cache representation unless it is a self-speculative path with a deliberately shared-compatible representation. Reusing cache tensors across these roles without an explicit ownership contract risks stale keys, invalid positions, or a prefix that no longer matches the committed token sequence.

This state also interacts with the cache-capacity accounting of Chapter 24. A request may temporarily need cache blocks for up to γ speculative positions even though it will ultimately retain fewer. Paged allocation can reduce the cost of such variable-length growth, but it does not remove the need to reserve, reclaim, or roll back blocks correctly. Quantized target weights can change the relative cost of verification and therefore the actual benefit of speculation; Chapter 25's model-weight optimization and speculative acceleration are complementary rather than automatically additive.

Under continuous batching, each active request may commit a different number of tokens after a verification pass. The scheduler in Chapter 26 must account for variable accepted lengths, target verification work, draft work, provisional cache growth, and the next iteration's available capacity. A policy that maximizes accepted tokens for one request can worsen TTFT or TPOT for others if it monopolizes a phase-specific microbatch. The proper evaluation unit is again an arrival mixture with declared latency and fairness objectives, not a single-request speedup.

8 Limits and Failure Modes

Speculative Decoding is most useful when ordinary Target Model Decode is expensive, a low-cost proposal mechanism has a reasonable acceptance rate, and the hardware can verify short candidate blocks efficiently. It may be ineffective when the Draft Model is expensive, the continuation is difficult to predict, request batches are already large enough to alter the bottleneck, or another constraint such as long-context cache traffic dominates. The method can increase total arithmetic work even when it reduces serial Target Model iterations; it uses concurrency to exchange some extra candidate computation for lower latency.

Common failures are diagnostic rather than mysterious. A low acceptance rate signals distribution mismatch, an inappropriate sampling transformation, or a draft path too weak for the workload. An excessive γ wastes proposal and verification work after early rejections. A large Draft Model can erase the latency benefit it was intended to create. Poor batching can leave short target verification passes underutilized, while overaggressive batching can damage interactive latency. Additional Draft Model weights, Draft Model KV Cache, provisional target-cache entries, and rollback bookkeeping can turn a nominal algorithmic gain into a memory-capacity regression.

Finally, exactness is conditional. It presumes that the Target Model distribution, tokenizer, context construction, positional indexing, logit transforms, random-number procedure, and residual distribution are implemented consistently. Approximate variants may be useful when a controlled quality trade-off is acceptable, but they should not be described as distribution preserving. Ordinary Decode remains preferable when its simple execution path better meets the workload's latency, memory, reproducibility, or operational constraints.

9 Implementation Contracts

The distribution contract must name the Target Model revision, Draft Model or draft path revision, tokenizer, vocabulary mapping, prompt construction, positional convention, temperature, filters, penalties, stop rules, and random-number semantics. Before every acceptance decision, the Target and Draft distributions must refer to the same conditional prefix and the same transformed token support. The implementation must define a safe action when $q_{i(\tilde{y}_i)} = 0$, when a transformed distribution has empty support, or when finite-precision arithmetic makes a probability ratio invalid.

The state contract must distinguish proposed, verified, accepted, rejected, corrected, committed, and discarded tokens. Target-side KV entries for a candidate block must be tagged provisional until the acceptance boundary is known. A rejection must reclaim later provisional entries exactly once, and a cancellation must reclaim both Target and Draft state. Tests should compare short seeded speculative samples against ordinary seeded Target Model samples under the same sampling policy, check that greedy outputs match exactly, and assert that a rejected suffix cannot influence later target logits.

The serving contract must record draft length, accepted-token count, rejection position, target-verification time, drafting time, cache reservation and rollback events, batch composition, and request-level TTFT and TPOT. It should report acceptance-rate distributions rather than only a global mean, because a long or difficult request can have behavior hidden by an aggregate. A deployment decision should compare end-to-end latency, throughput, cache capacity, and quality under the same model representation and scheduler policy as the ordinary Decode baseline.

10 Summary

Ordinary autoregressive generation requires one serial Target Model Decode step per output token. Speculative Decoding uses a cheap proposal mechanism to construct a short candidate continuation, then lets the Target Model verify that block in one causal pass. The Target Model remains authoritative: accepted proposals are committed only under its acceptance rule, and the first rejected proposal is replaced by a target-corrected token.

For stochastic generation, acceptance probabilities and residual sampling preserve the Target Model distribution when the implementation uses compatible transformed distributions. The potential gain is governed by accepted tokens per verification, Draft Model cost, draft length, verification efficiency, cache state, and serving policy. Self-speculation and MTP change how candidates are proposed, not the need to state whether target-side verification preserves the intended output behavior. Speculation is therefore a conditional systems optimization, not a replacement for ordinary Decode.

References

[1] Y. Leviathan, M. Kalman, and Y. Matias, "Fast Inference from Transformers via Speculative Decoding," in *Proceedings of the 40th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 19274–19286. [Online]. Available: https://proceedings.mlr.press/v202/leviathan23a.html

[2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating Large Language Model Decoding with Speculative Sampling," *arXiv preprint arXiv:2302.01318*, 2023, doi: 10.48550/arXiv.2302.01318.

[3] J. Zhang *et al.*, "Draft & Verify: Lossless Large Language Model Acceleration via Self-Speculative Decoding," *arXiv preprint arXiv:2309.08168*, 2023, doi: 10.48550/arXiv.2309.08168.

[4] F. Gloeckle, B. Youbi Idrissi, B. Roziere, D. Lopez-Paz, and G. Synnaeve, "Better & Faster Large Language Models via Multi-token Prediction," in *Proceedings of the 41st International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, pp. 15706–15734. [Online]. Available: https://proceedings.mlr.press/v235/gloeckle24a.html

[5] T. Cai *et al.*, "Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads," *arXiv preprint arXiv:2401.10774*, 2024, doi: 10.48550/arXiv.2401.10774.